

OBJECT ORIENTED PROGRAMMING...

Working with Classes, Constructors and Destructors





CONSTRUCTORS / DESTRUCTORS



ACTIVITY

Write a simple C++ class to represent a rectangle, including methods to calculate its perimeter and area

In Object-Oriented Programming (OOP), initializing variables when creating objects is generally a good practice.

Proper initialization ensures that the object is in a valid, predictable state from the moment it is instantiated.



CONSTRUCTORS

A **constructor** is a **special function** in a class that is **automatically called** when an object of the class is **created**.

CONSTRUCTORS

- Special class member functions **automatically called** when an object of a class is **created/instantiated**.
- Purpose: initialise the newly created object.
- A constructor's name: Same as the name of the class.
 - defined inside or outside the class definition
- A constructor CANNOT return a value and has NO return type (NOT even void).
- To create a constructor, use the same name as the class, followed by parentheses ():
- There are 3 types of constructors:
 - Default constructors
 - Parametrized constructors
 - Copy constructors

DEFAULT CONSTRUCTOR: EXAMPLE

The constructor which does NOT take any argument (has no parameters).



ACTIVITY

Write a default constructor to initialize the variables to Zero.

DEFAULT CONSTRUCTOR: EXAMPLE

The constructor which does NOT take any argument (has no parameters).

Example:

```
Rectangle()
{
    width=0;
    length=0;
}
```

DEFAULT CONSTRUCTOR

- A constructor that can be invoked **without any arguments** e.g. a constructor with an **empty parameter list**, or a constructor with **default values** for all of its arguments.

ACTIVITY:

Is it possible to define more than one default constructor for a class? [YES/NO]
Justify...

Even if any constructor is NOT defined explicitly, the compiler will automatically provide a default constructor implicitly that leaves all data members un-initialized.

PARAMETERIZED CONSTRUCTOR: EXAMPLE

Pass arguments to constructors. The initial values are passed as arguments to the constructor function.

The arguments passed to a constructor are typically used to initialise the object's data members

Example:

```
Rectangle(int w, int l)
{
    width=w;
    length=l;
}
```

Used to initialize the various data elements of different objects with different values when they are created.

INVOKE CONSTRUCTORS

- A constructor CANNOT be called directly like a normal member function.

Example:

```
int main()
{
    Rectangle rect1;
        rect1.Rectangle();
}
```



PASSING ARGUMENTS TO CONSTRUCTOR

- Constructor is not invoked like a normal function.
- Arguments can **only** be passed to it when an object is declared.
- The *comma-separated list* of arguments must appear in parentheses after the object name.

Example:

```
int main()
{
    Rectangle rect1(1,1);
}
```

CONSTRUCTORS

Whenever one or more non-default constructors (with parameters) are defined for a class ...

A default constructor(without parameters) should be explicitly defined as the compiler will not provide a default constructor.

CONSTRUCTORS VS. MEMBER FUNCTIONS

Differences

- **Constructor** name must be same as class name but **Functions** cannot have same name as class name.
- **Constructor** does not have a return type but **Functions** must have a return type.
- **Constructors** are invoked automatically at the time of object creation and cannot call explicitly using class objects. **Functions** are called explicitly using class objects.

Similarities

- **Constructors** and **Member Functions** can be defined with different parameters.
- Parameters can be passed into **Constructors** and **Member Functions** as arguments.



RECAP

From Last Week ...

SCS1310

DEFINING CONSTRUCTOR VS MEMBER FUNCTION

Constructor Syntax:

```
className :: className(... )  
{  
    ...  
}
```

Member Function Syntax:

```
ReturnType className::MemberFunctionName(... )  
{  
    .....  
}
```

CONSTRUCTOR INITIALIZATION

//A constructor initializes variables to zero

```
Rectangle::Rectangle()
{
    width=0;
    length=0;
}
```

//A parameterized constructor

```
Rectangle::Rectangle(int w, int l)
{
    width=w;
    length=l;
}
```

//A constructor initializes variables to zero

```
Rectangle::Rectangle(): width(0),length(0)
{ }
```

//A parameterized constructor

```
Rectangle::Rectangle(int w, int l): width(w),length(l)
{ }
```

CONSTRUCTOR INITIALIZATION LIST

- A list in the definition of a constructor that specifies initial values for the data members of that class.
- Initialise its data members by invoking constructors

```
ClassName(parameterList) : member1(valueList1),member2(valueList2)...  
{  
    // ...  
}
```

OVERLOADED CONSTRUCTOR

- Different functions can have the same name as long as they differ in their parameter lists.
- Known as Overloading
- Constructors are often overloaded to provide alternative ways of creating an instance of a class.

//Constructor signature

- ① **Rectangle();**
- ② **Rectangle(int,int);**
- ③ **Rectangle(int);**



DESTRUCTOR

A special member function that is automatically called whenever an object of its class ceases to exist.

DESTRUCTORS

- Destroys or deletes an object.
- Release any resources that the corresponding constructor allocated.
- A destructor's name is the same as the name of the class it belongs to, except that the name is preceded by a tilde symbol (~).

Syntax:

```
~constructor-name();
```

DESTRUCTOR: EXAMPLE

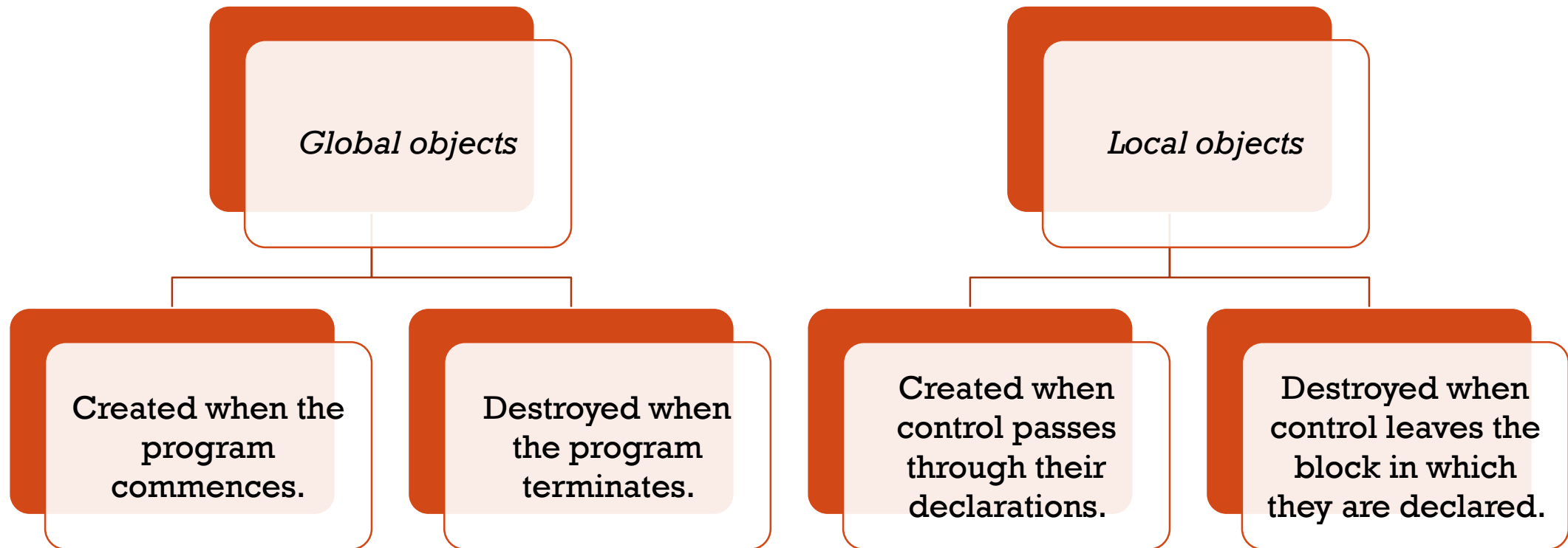
```
Constructor is called  
Hello World!  
Destructor is called
```

```
1  #include <iostream>
2  using namespace std;
3
4  class HelloWorld{
5  public:
6      HelloWorld(){
7          cout<<"Constructor is called"<<endl;}
8
9      ~HelloWorld(){
10         cout<<"Destructor is called"<<endl;}
11
12         void display(){
13             cout<<"Hello World!"<<endl;
14         }
15     };
16
17 int main(){
18     //Object created
19     HelloWorld obj;
20     //Member function called
21     obj.display();
22 }
```

DESTRUCTORS

- Should begin with tilde sign(~)
- It cannot be declared static or const.
- The destructor does not have arguments.
- It has no return type, not even void.
- A destructor should be declared in the public section of the class.
- There cannot be more than one destructor in a class.
- The compiler generates a default destructor when a destructor is NOT specified in a class.

OBJECT LIFETIMES VS CONSTRUCTOR/DESTRUCTOR CALL



CONSTRUCTORS / DESTRUCTORS: CALL ORDER

- Depends on order of execution (when execution enters and exits scope of objects)
- Destructor calls reverse order of constructor calls
- Global scope objects
 - Constructors are called before any other function (including main)
 - Destructors are called when main terminates (or exit function called)
 - Not called if the program terminates with abort
- Local scope objects
 - Constructors are called when objects are defined and each time execution enters the scope
 - Destructors are called when objects leave scope / execution exits block in which object defined
 - Not called if the program ends with abort



ACTIVITY

OUTPUT?

SCS1310

```

1  #include<iostream>
2  using namespace std;
3
4  class Rectangle
5  {
6      int width;
7      int length;
8
9      public:
10
11      1 Rectangle()
12      {
13          width=0;
14          length=0;
15      }
16
17      2 Rectangle(int w,int l){
18          width=w;
19          length=l;
20      }
21
22      3 Rectangle(int w){
23          width=w;
24          length=0;
25      }
26
27      ~Rectangle(){
28          cout<<"Destructor is called"<<endl;
29          cout<<"Object with "<<width<<" "<<length<<endl;
30      }
31
32      void getWL();
33  };
34

```

```

35 void Rectangle::getWL()
36 {
37     cout<<"Width is "<<width<<endl;
38     cout<<"Length is "<<length<<endl;
39 }
40
41 int main()
42 {
43     Rectangle Rec3;
44     Rec3.getWL();
45
46     Rectangle Rec1(4);
47     Rec1.getWL();
48
49     Rectangle Rec2(1,1);
50     Rec2.getWL();
51 }
52

```

```

Width is 0
Length is 0
Width is 4
Length is 0
Width is 1
Length is 1
Destructor is called
Object with 1 1
Destructor is called
Object with 4 0
Destructor is called
Object with 0 0

```

```
class Rectangle  
{  
    int width;  
    int length;  
    public:  
        //constructor signature  
        Rectangle();  
        Rectangle(int,int);  
        Rectangle(int);  
        //destructor signature  
        ~Rectangle();  
        //function signature  
        void getWL();  
        int areaRect();  
};
```



ACTIVITY: RECTANGLE 2

```

1  #include<iostream>
2  using namespace std;
3
4  class Rectangle
5  {
6      int width;
7      int length;
8
9      public:
10         //constructor signature
11         Rectangle();
12         Rectangle(int,int);
13         Rectangle(int);
14         //destructor signature
15         ~Rectangle();
16         //function signature
17         void getWL();
18         int areaRect();
19     };

```

```

42  //A function to calculate the area of the rectangle
43  int Rectangle::areaRect()
44  {
45      return width*length;
46  }
47
48  int main()
49  {
50      Rectangle Rec1(4);
51      Rec1.getWL();
52
53      Rectangle Rec2(1,1);
54      Rec2.getWL();
55
56      Rectangle Rec3;
57      Rec3.getWL();
58
59      cout<<"Rectangle 1 Area: "<<Rec1.areaRect()<<endl;
60      cout<<"Rectangle 2 Area: "<<Rec2.areaRect()<<endl;
61      cout<<"Rectangle 3 Area: "<<Rec3.areaRect()<<endl;
62  }

```

```

Width is 4
Length is 10
Width is 1
Length is 1
Width is 0
Length is 0
Rectangle 1 Area: 40
Rectangle 2 Area: 1
Rectangle 3 Area: 0
Destructor is called: 0
Destructor is called: 1
Destructor is called: 40

```



ACTIVITY :

```

1  #include<iostream>
2  using namespace std;
3
4  class CreateDestroy{
5      private:
6          int objectID;
7          string message;
8
9      public:
10         CreateDestroy():objectID(0),message(""){}
11         CreateDestroy(int, string);
12         ~CreateDestroy();
13     }obj;
14
15     CreateDestroy obj123(123,"Global Object");
16

```

```

17 CreateDestroy::CreateDestroy(int ID,string msg){
18     objectID = ID;
19     message=msg;
20     cout << "Object " << objectID << " constructor runs "
21         << message << endl;
22 }
23
24 CreateDestroy::~~CreateDestroy()
25 {
26     cout << "Object " << objectID << " destructor runs"
27         << message << endl;
28 }
29

```

```

30 int main()
31 {
32     cout << "MAIN FUNCTION: EXECUTION BEGINS" << endl;
33     CreateDestroy obj1( 1, "(local automatic in main)" );
34     CreateDestroy obj2( 2, "(local automatic in main)" );
35     CreateDestroy obj3( 3, "(local automatic in main)" );
36     cout << "\nMAIN FUNCTION: EXECUTION ENDS" << endl;
37 }

```

```
Object 123 constructor runs Global Object  
MAIN FUNCTION: EXECUTION BEGINS  
Object 1 constructor runs (local automatic in main)  
Object 2 constructor runs (local automatic in main)  
Object 3 constructor runs (local automatic in main)  
  
MAIN FUNCTION: EXECUTION ENDS  
Object 3 destructor runs(local automatic in main)  
Object 2 destructor runs(local automatic in main)  
Object 1 destructor runs(local automatic in main)  
Object 123 destructor runsGlobal Object  
Object 0 destructor runs  
  
-----  
Process exited after 1.017 seconds with return value 0  
Press any key to continue . . . |
```



INCLASS ACTIVITY :

Write down the output...

```
#include<iostream>
#include<cassert>
using namespace std;

class List{
    private:
        int length;
        int *array;
    public:
        List();
        List(int size);
        ~List();

        void displayList();
        void SetList(int a);
};
```

```
int main()
{
    cout<<"START"<<endl;

    List l1;
    l1.displayList();
    l1.SetList(2);
    l1.displayList();

    List l2(3);
    l2.displayList();
}
```

```

List::List()
{
    length = 10;
    array = new int [length];
    assert(array != 0);
    for (int i = 0; i < length; i++)
        array[i] = 0;
}

List::List(int size)
{
    length = size;
    if (length <= 0)
        array = 0;
    else
    {
        array = new int [length];
        assert(array != 0);
        for (int i = 0; i < length; i++)
            array[i] = 3;
    }
}

```

```

void List::displayList(){
    for (int i = 0; i < length; i++)
        cout<<array[i]<<",";
    cout<<endl;
}

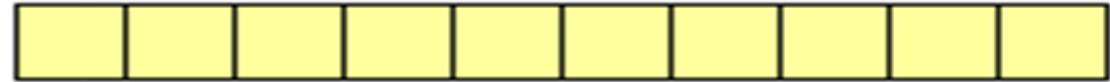
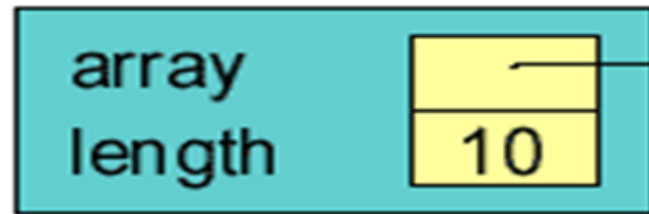
void List::SetList(int a){
    for (int i = 0; i < length; i++)
        array[i] = i+a;
}

// Destructor
List::~~List()
{
    delete [] array;
    cout<<"Called the destruct"<<endl;
}

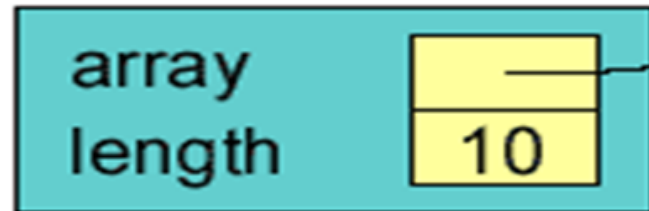
```

COPY OBJECTS

aList



Bitwise copy of aList





DEFAULT COPY CONSTRUCTOR

Makes a bitwise copy of the class instance

COPY CONSTRUCTOR

- An overloaded constructor that initializes an object using another object of the same class.
- Initialize the members of a newly created object by copying the members of an already existing object.
- A copy Constructor may be called, when
 - an object of the class is returned by value.
 - an object of the class is passed (to a function) by value as an argument.
 - an object is constructed based on another object of the same class.
 - the compiler generates a temporary object.
- A copy constructor cannot return a value and has no return type (not even **void**).
- Allows instantiating an object as an exact copy of another in terms of its attribute values.
- Default or User-Defined copy constructor

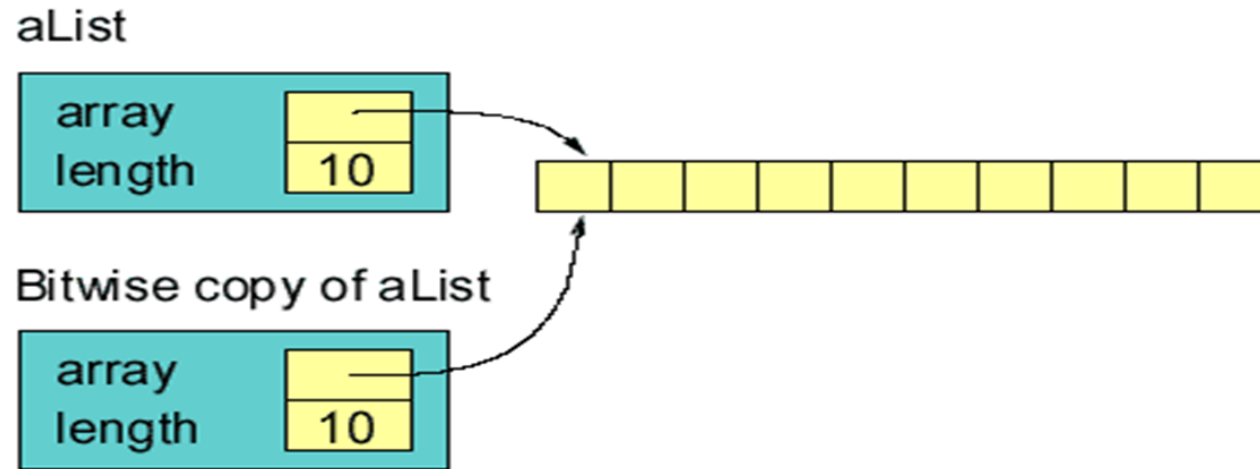
DEFAULT COPY CONSTRUCTOR

If a copy constructor for a class is NOT defined explicitly, then the compiler will *automatically* provide a copy constructor that makes a *bitwise copy* of the class instance.

Bitwise copying is a way to get a copy of a class object by copying every bit (byte) of a particular class object (instance). Bitwise copying is used when it is necessary to obtain an identical destination object from the source object.

DEFAULT VS USER DEFINED COPY CONSTRUCTOR

- Works fine in general.
- HOWEVER, if an object has pointers or any runtime allocation of the resource like a file handle, a network connection, etc.
- Need to define a user-defined copy constructor.



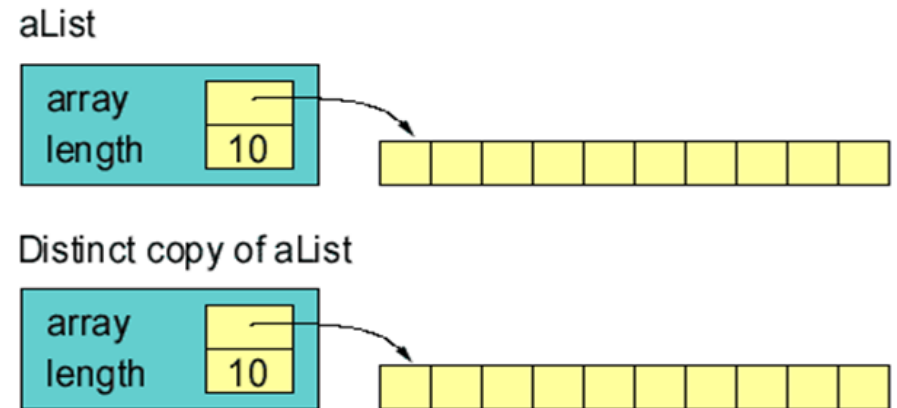
USER DEFINED COPY CONSTRUCTOR

Copy Constructor Syntax

```
className::className(const className& old_Object)
{
    // ...
}
```

USER DEFINED COPY CONSTRUCTOR

```
List::List(const List &original)
{
    length = original.length;
    if (length > 0)
    {
        array = new int [length];
        assert(array != 0);
        for(int i = 0; i < length; i++)
            array[i] = original.array[i];
    }
    else
        array = 0;
}
```



46

**WHY ARGUMENT TO A COPY
CONSTRUCTOR MUST BE
PASSED AS A REFERENCE?**



ACTIVITY 01: COPY CONSTRUCTOR

```
#include <iostream>
using namespace std;

class Sample {
    int id;

public:
    void init(int x) { id = x; }
    void display() { cout << endl << "ID=" << id; }
};

int main()
{
    Sample obj1;
    obj1.init(10);
    obj1.display();

    Sample obj2(obj1);
    obj2.display();
    return 0;
}
```



ACTIVITY 02 : COPY CONSTRUCTOR

```
#include <iostream>
using namespace std;

class Sample {
    int *id;

public:
    void init(int x) {
        if (id == NULL) {
            id = new int; // Allocate memory
        }
        *id = x;
    }

    void display() {
        cout << endl << "ID = " << *id;
    }
};
```

```
int main() {
    int x = 100;

    Sample obj1;
    obj1.init(x);
    obj1.display();

    Sample obj2(obj1);
    obj2.display();

    int y = 200;
    obj2.init(y);
    obj1.display();
    obj2.display();

    return 0;
}
```



ACTIVITY

```

1  #include<iostream>
2  using namespace std;
3
4  class Hello{
5      string name;
6
7      public:
8
9      Hello();
10     Hello(string);
11
12     void display();
13
14     ~Hello(){
15         cout<<"Destructor is called: " <<name<<endl;}
16 };
17
18 Hello::Hello(){
19     name=" ";}
20
21 Hello::Hello(string n){
22     name= n;}
23
24 void Hello::display(){
25     cout<<"constructor is called and initialized the variable to: "<<name<<endl;}
26

```

```

27 int main()
28 {
29     Hello H1, H2("My Name");
30
31     Hello H3=H2;
32
33     H1.display();
34     H2.display();
35     H3.display();
36     H2.display();
37 }

```

```

constructor is called and initialized the variable to:
constructor is called and initialized the variable to: My Name
constructor is called and initialized the variable to: My Name
constructor is called and initialized the variable to: My Name
Destructor is called: My Name
Destructor is called: My Name
Destructor is called:

```



ACTIVITY

```

class Engine{
    private:

        int cylinders; // Number of cylinders
        double liters; // Capacity in liters

    public:

        Engine(int c = 4, double l = 1.3); // Constructor
};

Engine::Engine(int c, double l) : cylinders(c), liters(l)
{
    cout<<"Constructor Engine called "<<endl;
    cout<<"Cylinders: "<<cylinders<<endl;
    cout<<"Liters: "<<litres<<endl;
}

```

```

class Car
{
    private:
        int modelNumber; // Model number
        Engine motor;     // Car's engine

    public:
        // Constructor
        Car(int m, int c, double l);
};

Car::Car(int m, int c, double l):modelNumber(m),
motor(c, l)
{
    cout<<"Constructor Car called"<<endl;
    cout<<"Model No: "<<modelNumber<<endl;
}

int main(){
    Car c(2,3,2.3);
}

```

```
class Engine{  
    private:
```

```
class Car  
{  
    private:
```

Constructor Engine finished initializing the variables

Cylinders: 3

Liters: 2.3

Constructor Car finished initializing the variables

Model No: 2

Process exited after 0.1143 seconds with return value 0

Press any key to continue ...

```
int main(){  
    Car c(2,3,2.3);  
}
```

```

class Engine{
    private:

        ⑥ int cylinders; // Number of cylinders
        ⑦ double liters; // Capacity in liters

    public:

        Engine(int c = 4, double l = 1.3); // Constructor
};

5 Engine::Engine(int c, double l) : cylinders(c),
  liters(l)
{ ⑧
    cout<<"Constructor Engine finished initializing the
variables"<<endl;
    cout<<"Cylinders: "<<cylinders<<endl;
    cout<<"Liters: "<<litres<<endl;
} ⑨

```

```

class Car
{
    private:
        ④ int modelNumber; // Model number
        ⑤ Engine motor; // Car's engine

    public:
        // Constructor
        Car(int m, int c, double l);
};

3 Car::Car(int m, int c, double l):
  modelNumber(m), motor(c, l)
{ ⑩
    cout<<"Constructor Car finished initializing the
variables"<<endl;
    cout<<"Model No: "<<modelNumber<<endl;
}

1 int main(){
    2 Car c(2,3,2.3);
}

```



DEFINING CLASS OBJECTS WITHIN A CLASS

Example: Creating a Cylinder

A SIMPLE CLASS

```
class Circle{  
    double radius;  
  
    public:  
        Circle(){ radius=0.0;}  
        double area();  
};
```

In OOP

- Class is the blueprint /template
- Use the template to create Objects
- Class includes class variables
 - Also known as data members / member variables
- Class Constructors
 - Executes / invoked when an instant is created using the class /create a new instance of a class.
- Member Functions
 - Get different functionalities

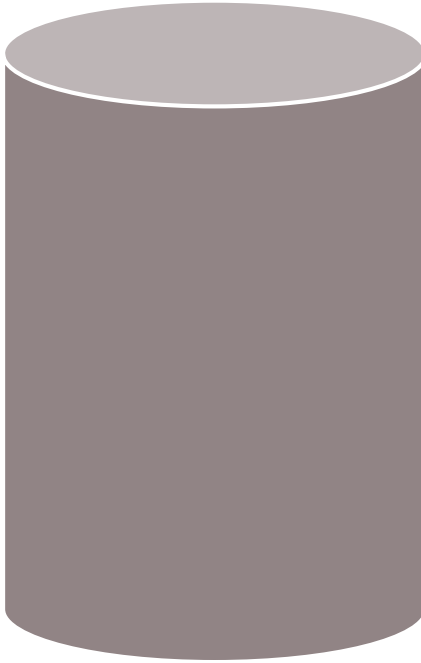
DEFINING CONSTRUCTORS

```
Circle ()  
{  
    radius = 0;  
}
```

```
Circle ():radius (0)  
{ }
```

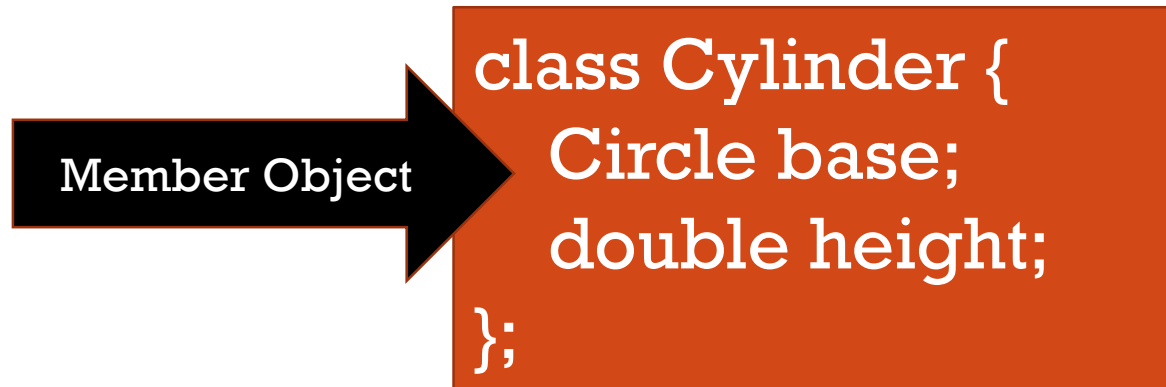
```
Circle (int r)  
{  
    radius = r;  
}
```

```
Circle (int r):radius (r)  
{ }
```

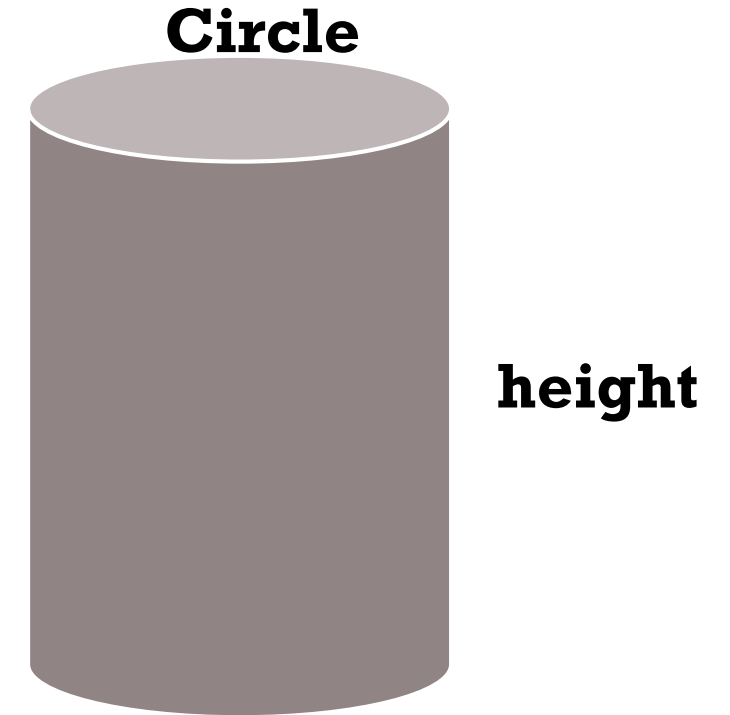


Cylinder

DEFINING CLASS OBJECTS WITHIN A CLASS



class Cylinder has a member object whose type is another class (In this example: Circle)



DEFINING CLASS OBJECTS WITHIN A CLASS

```
class Circle{
    double radius;
    public:
        Circle():radius(0) {
            cout<<"leaving the circle constructor" <<endl;
        }
        Circle (double r):radius(r){
            cout<<"leaving the Circle constructor: Radius
passed as a variable"<<endl;
        }
        double area();
};
```

```
class Cylinder {
    Circle base;
    double height;
    public:
        Cylinder(double r, double h) : height(h), base(r)
        {
            cout<<"leaving the Cylinder constructor"<<endl;
        }
        double volume() {return base.area() * height;}
};
```

DEFINING CLASS OBJECTS WITHIN A CLASS

1. Create a class object “Cyl1” of the type “Cylinder”.
2. The constructor will be called.
`Cylinder(double h, double r) : height(h), base(r)`
3. Create a double variable named height and assign it a value of h.
4. Create a class object name “base” and call the corresponding constructor to initialize the variables of object “base”.
5. The Circle constructor will be called.
`Circle (double r):radius(r){
cout<<"leaving the Circle constructor: Radius passed as a variable"<<endl; }`
6. End of the Circle constructor.
7. End of the Cylinder constructor.
8. End of the main() function.

`Cylinder Cyl1(3,4);`

STATIC KEYWORD IN C++

- Allocate storage only once in a program lifetime in static storage area.
- The scope exists till the program ends (program lifetime).
- Static Keyword can be used to define,
 - Variables in functions
 - Class Objects
 - Member Variables in class
 - Methods in class

STATIC VARIABLES IN A FUNCTION

- The variables declared as static are initialized only once.
- They are allocated space in separate static storage.
- Space for **it gets allocated for the lifetime of the program.**
- The value of variable in the previous call gets carried through the next function call.
- Even if the function is called multiple times, space for the static variable is allocated only once

EXAMPLE

```
0 0
1 0
2 0
3 0
4 0

-----
Process exited after 0.1813 seconds with return value 0
Press any key to continue . . .
```



```
void demo()
{
    static int count = 0;
    int c = 0;

    cout << count << " " << c << endl;

    count++;
    c++;
}

int main()
{
    for (int i=0; i<5; i++)
        demo();
}
```

STATIC VARIABLES IN A CLASS

- The static variables **in a class are shared by the objects.**
- Can not have multiple copies of the same static variables for different objects.
- Can not be initialized using constructors, as these are not dependent on object initialization
- A static variable inside a class should be initialized explicitly, always outside the class using the class name and scope resolution operator outside the class.

EXAMPLE:

Type ClassName::variable_name = value;

ACCESSING STATIC VARIABLES IN A CLASS

- A member function can refer to a static data member of the same class directly.
- A non-member function can refer to a static data member using either the notation

`ClassName::staticMemberName` or
`objectName.staticMemberName`

STATIC CLASS OBJECTS

- Like variables, objects also when declared as static have a scope till the lifetime of the program.
- Exists the scope at the end of the program.

EXERCISE

```
int main()
{
    int x=0;
    if(x==0){
        static StaticObject obj;
    }
    cout << "End of main\n";
}
```

```
class StaticObject
{
    int i;
public:
    StaticObject():i(0)
    {
        cout << "Inside Constructor"<<endl;
    }

    ~StaticObject()
    {
        cout << "Inside Destructor\n";
    }
};
```

STATIC MEMBER FUNCTIONS

- Work for the class as a whole rather than for a particular object of a class.
 - Does not depend on the object of a class.
- It can be called using an object and the direct member access “.” operator.
 - Recommended invoking the static members using the class name and the scope resolution operator.
- Allowed to access only the static data members or other static member functions
 - Can not access the non-static data members or member functions of the class.

STATIC MEMBER FUNCTIONS

- Like static data members, static member functions are associated with a class and not with any particular object of that class.
- Static member functions do not have a **this** pointer.
- Static member functions cannot be declared **const**.
- A member function can call a static member function of the same class directly.
- A non-member function can call a static member function using either the below notation

`ClassName::staticMemberName()` or
`objectName.staticMemberName()`

```

1  #include <iostream>
2  using namespace std;
3
4  class Counter {
5      static int count;
6
7  public:
8      static void increment() {
9          count++;
10     }
11
12     static void display() {
13         cout << "Current count is: " << count << endl;
14     }
15 };

```

```

17 int Counter::count = 0;
18
19 int main() {
20     Counter::display();
21     Counter::increment();
22     Counter::increment();
23     Counter::display();
24
25     Counter c1, c2;
26     c1.increment();
27     Counter::display();
28
29     return 0;
30 }

```

```

Current count is: 0
Current count is: 2
Current count is: 3

```

```

-----
Process exited after 1.031 seconds with return value 0
Press any key to continue . . .

```

CONSTANTS

- Variables
- Pointers
- Function arguments and return types
- Class Data members
- Class Member functions
- Objects

CONSTANT VARIABLES IN C++

- Cannot change its value of the variable.
- Value of the variable will not change during its lifetime.
- Constant variables must be initialized while they are declared.

```
int main()
{
    const int i = 10;
    const int j = i + 10;
    i++;
}
```



Compilation error

POINTERS WITH CONST KEYWORD

Declaration can be done in TWO ways.

- Pointer to a constant variable
 - The pointer is pointing to a const variable.
- Constant Pointer
 - Cannot change the pointer, to which it points to
 - Can change the value that it points to, by changing the value of the variable.
 - Useful when one want to change the value but not the memory.
 - The pointer will always point to the same memory location, as it is defined with const keyword, but the value at that memory location can be changed.

```
const int* u;
```

```
int x = 1;  
int* const w = &x;
```

```
#include <iostream>
using namespace std;
```

Compile?

```
int main() {
    int a = 10;
    int b = 20;
    const int c = 30;

    const int* ptrToConst = &a;
    cout << "ptrToConst points to a = " << *ptrToConst << endl;

    ptrToConst = &c;
    cout << "ptrToConst now points to c = " << *ptrToConst << endl;

    int* const constPtr = &a;
    *constPtr = 25;
    cout << "constPtr points to a with new value = " << *constPtr << endl;

    const int* const constPtrToConst = &c;
    cout << "constPtrToConst points to c = " << *constPtrToConst << endl;
    return 0;
}
```

CONSTANT FUNCTION ARGUMENTS AND RETURN TYPES

- The return type or arguments of a function can be defined as constant.
- Member functions should not modify member data.

```
void func(const int i)
{
    i++; // error
}

const int funcCont()
{
    return 6;
}
```

DEFINING CLASS DATA MEMBERS AS CONSTANT

- Data members are not initialized during declaration.
- The initialization is done in the constructor.
- Once initialized cannot change the value.

Exercise:

Write a function called setValue within the class “TestConst” and try to change the variable ‘var’.

Compiles correctly

```
class TestConst
{
    const int var;
public:
    TestConst(int x):var(x)
    {
        cout << "Const Var: " <<
var<<endl;
    }
};

int main()
{
    TestConst testObj1(3);
    TestConst testObj2(2);
}
```

DEFINING CLASS OBJECT AS CONSTANTS

- During the object lifetime Data members can never be changed.

```
const class_name object;
```

CONSTANT OBJECTS

- Objects are seldom passed by value in C++.
- If you want to pass objects, they are passed by passing a **const** reference
- Passing by **const** reference
 - Avoids copying the object
 - Protects the object from unintentional changes.

```
void displayTime(const Time &t)
```

DEFINING CLASS MEMBER FUNCTION AS CONSTANTS

- **const keyword** must be added to both the function signature and implementation.
- A constant member functions never modifies data members in an object.

```
returnType functionName() const
```

```
int displayList() const;
```

```
int List::displayList () const  
{  
    .....  
}
```

86

'THIS' KEYWORD

It points to the **object** that is currently invoking the method

Rectangle.cpp

SCS1310

THE “THIS” POINTER IN C++

- How do objects work with functions and data members of a class?
- When an object is created
 - Each object gets its own copy of the data member.
 - All-access the same function definition as present in the code segment.
- Each object gets its own copy of data members and share a single copy of member functions.
- How the proper data members are accessed and updated?
 - An implicit pointer ‘this’ (value is the address of the object that generated the call)
 - Passed as a hidden argument to all non-static member function calls and
 - Available as a local variable within the body of all non-static functions.
 - ‘this’ pointer is not available in static member functions as static member functions can be called without any object (with class name).

IN RECTANGLE EXAMPLE...

- **this** points to the object that invoked **getWL()**.

```
void Rectangle::getWL()
{
    cout<<"Width is "<<this->width<<endl;
    cout<<"Length is "<<this->length<<endl;
}
```



EXERCISE

```
#include<iostream>
using namespace std;

class Counter {
    int value;
public:
    Counter(int v = 0) : value(v) {}
    Counter& increment() {
        value++;
        return *this; // returns current object by reference
    }
    void display() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Counter c;
    c.increment().increment().increment();
    c.display();
}
```

HOMEWORK

Friend Functions(What, Why & How)

Forum post discussion

